

Week 2 · Laboratory Problems — build the PID by hand in Simulink

- Course: USV Guidance, Navigation and Control (Graduate) · Department of Autonomous Vehicle System Engineering
- Time: **one hour**, immediately after the Week 2 lecture hour
- Three problems, in order. Each one adds to the previous answer.

What this hour is for

The lecture measured what P, I and D do on one mass-spring-damper, with a model that was generated for it. This hour builds the controller by hand, block by block, until the model reproduces the lecture's numbers. A PID that has been assembled from gains and integrators can be read; a PID block that has only been double-clicked cannot.

The setpoint, the plant and the log are provided. The controller is the exercise.

Before starting

```
cd lectures/W02_simulink/problems
W02_P1_start          % creates W02_P1.slx – setpoint, plant and log only
W02_check(1)          % run this whenever, as often as needed
```

The solver is already set to fixed-step `ode4` at `h = 1` ms. **Do not change it.**

★ Two requirements on every model

- The To Workspace block `ylog` stays as provided. Its Mux takes y_d on input 1, the force τ on input 2, and y on input 3. Input 2 is the one wire the student adds.
- Every gain is written as a **variable name** — `Kp`, `Ki`, `Kd`, `Nf`, `tau_max`, `Kb` — never as a number. `W02_check` sets those names itself.

Problem 1 · Proportional only (15 minutes)

Build. Error, one gain, into the plant.

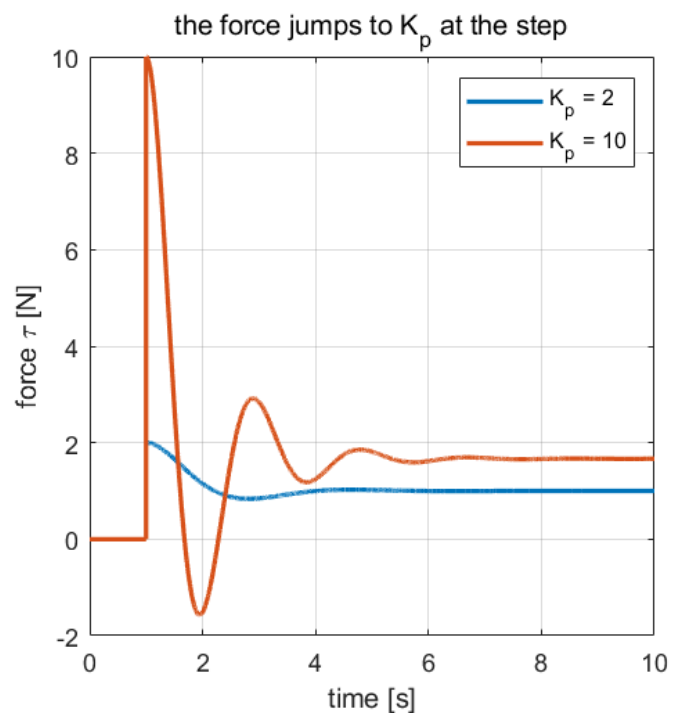
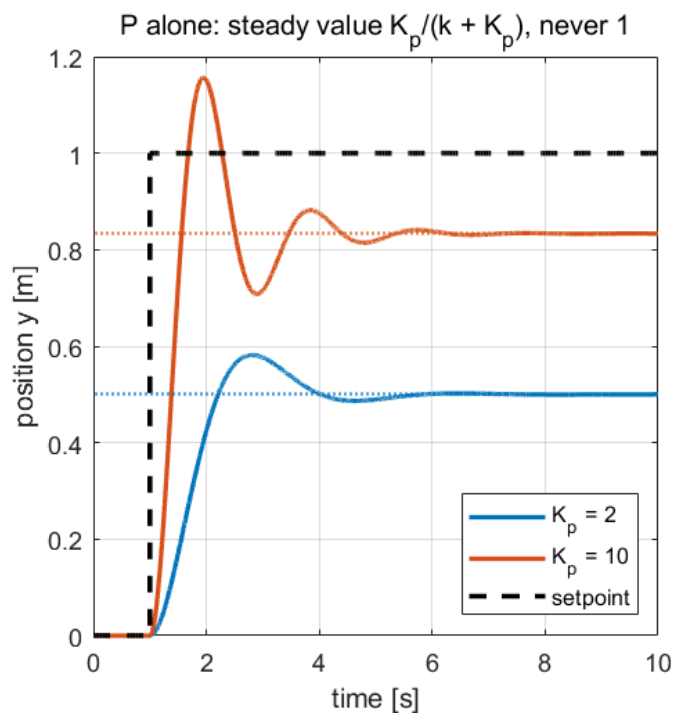
```
setpoint → (+)(-) → Kp → τ → plant → y
           ↑           |
           └──────────┘
```

Predict before running. From §2-3, $y_{ss} = K_p / (k + K_p)$ and $\zeta = b / (2\sqrt{m(k + K_p)})$ with $m = 1$, $b = 2$, $k = 2$. Write down both numbers for $K_p = 2$ and $K_p = 10$ **before** pressing Run.

Verify. `W02_check(1)`.

K_p	steady value	overshoot
2	0.500	16.3 %
10	0.833	38.8 %

What a correct model produces



Reading the figure	
left	both gains settle on their dotted line $K_p/(k + K_p)$, never on the setpoint
right	the force jumps to K_p newtons at the step, then settles on ky_{ss}
the check	if the response settles on 1, the loop is not proportional-only — check that nothing else is summed into τ

The point. The spring needs a steady force ky , and a proportional controller makes force only from error, so some error must remain. No value of K_p removes it.

Problem 2 · Add I and D, by hand (25 minutes)

Build. Two more branches, from **Gain, Sum and Integrator blocks only**. No PID Controller block, no Derivative block — the checker looks for both.

$$\tau = K_p e + K_i \int e dt + d, \quad d = N_f (K_d e - x), \quad \dot{x} = d$$

⚠ The derivative is filtered, and it is built as a loop

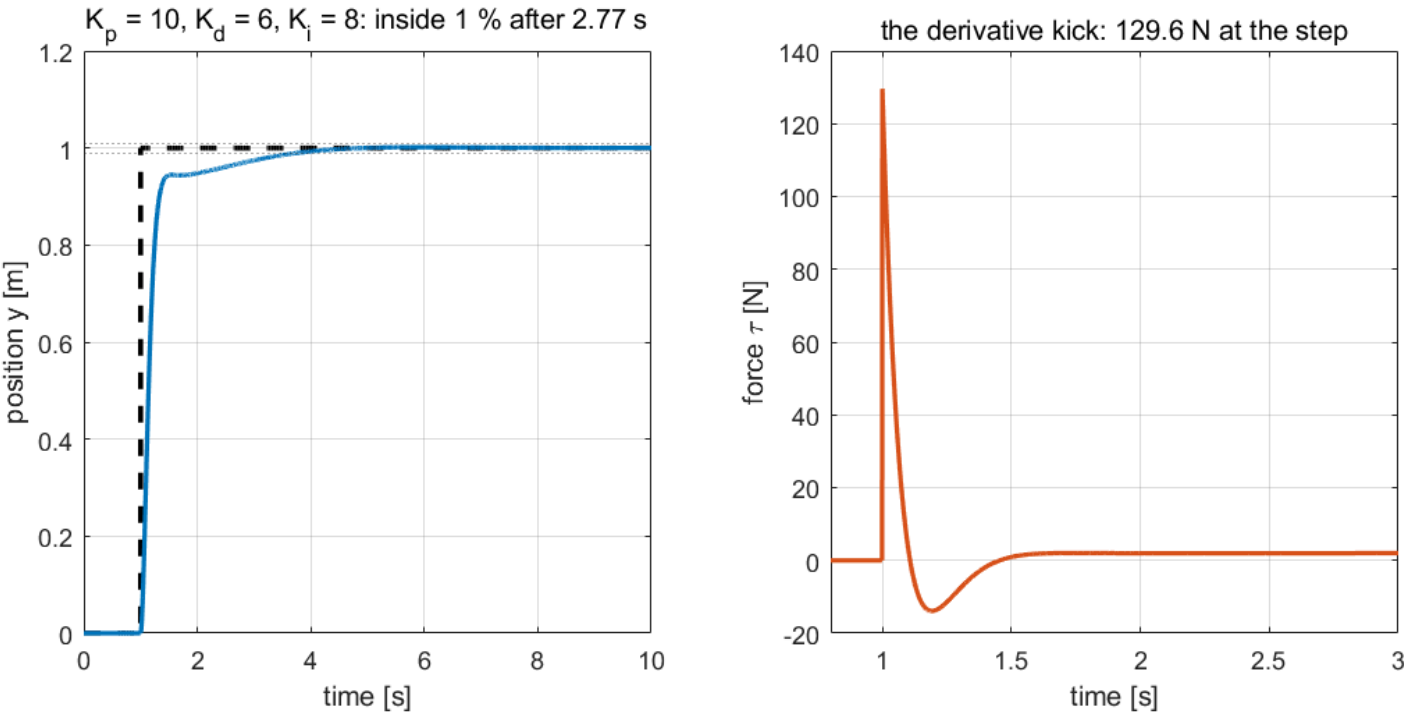
The second line is $K_d N_f s / (s + N_f)$ applied to e (§2-7). Build it as a gain N_f with an integrator in its feedback path, exactly as box D of [w02_pid.slx](#). A Derivative block would differentiate the corner of the step into an impulse.

Predict before running. These are the gains the tuning order of §2-9 arrived at: $K_p = 10$, $K_d = 6$, $K_i = 8$, $N_f = 20$. From §2-7, what is the force at the instant of the step?

Verify. [w02_check\(2\)](#).

Quantity	Lecture value
overshoot	0.16 %
inside 1 % of the setpoint after	2.77 s
largest force	129.6 N

What a correct model produces



Reading the figure	
left	the response enters the dotted 1 % band and stays: the integral has removed the error
right	the force at the step, $K_d N_f + K_p \approx 130$ N: the derivative kick
the check	an overshoot near 8.6 % means K_i is not reaching the sum; a force of thousands of newtons means the derivative is not filtered

The point. Every block of the model is one symbol of the equation above. The PID block's dialog — P, I, D, N — is the same four numbers, and nothing else.

Problem 3 · A limit on the force, and anti-windup (20 minutes)

Build. A Saturation block on τ , limits `-tau_max` and `tau_max`, and one more input to the integrator:

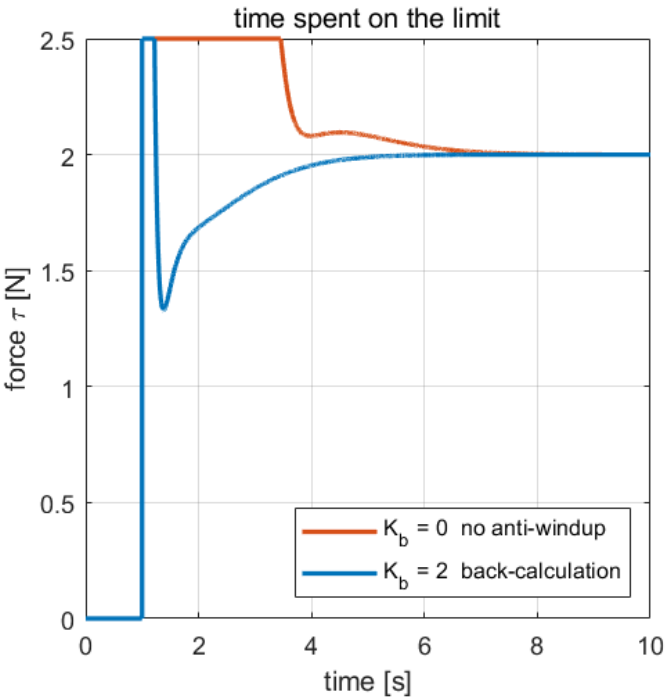
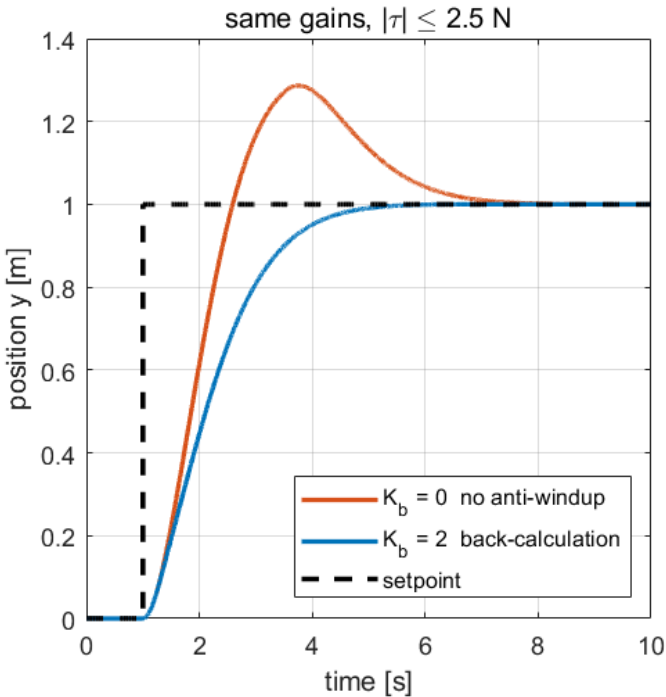
$$\dot{I} = K_i e + K_b (\tau - u), \quad u = P + I + D, \quad \tau = \text{sat}(u)$$

Predict before running. The checker sets $|\tau| \leq 2.5$ N. Holding $y = 1$ against the spring needs 2 N, so the target is reachable. With $K_b = 0$, what happens to the integrator while the force sits on the limit?

Verify. `w02_check(3)`, gains $K_p = 10, K_i = 8, K_d = 4$.

	overshoot	settling (2 %)
$K_b = 0$	28.67 %	5.54 s
$K_b = 2$	0.03 %	3.56 s

What a correct model produces



Reading the figure	
left	the same gains and the same limit; only the orange run overshoots
right	the force: orange sits on 2.5 N for more than two seconds, blue leaves the limit almost at once
the check	if the two runs are identical, $\tau - u$ is not reaching the integrator, or Kb is written as a number

The point. Without anti-windup the integrator stores what the actuator could not deliver and pays it back as overshoot. Back-calculation tells it how much was cut off.

Marking

	Weight	What is being marked
Problem 1	25	<code>w02_check(1)</code> passes; why $y_{ss} \neq 1$ is stated in one sentence
Problem 2	45	<code>w02_check(2)</code> passes, with no PID or Derivative block; the force at the step is predicted before it is measured
Problem 3	30	<code>w02_check(3)</code> passes; the role of $\tau - u$ is explained